

Terraform Modules

Package infrastructure into reusable building blocks — the DRY principle applied to your whole cloud.

What is a module

Structure

Registry & sharing

Versioning

Lab 09

Today's roadmap



1 • What & why

Modules, benefits, and the orchestrator pattern.



2 • Structure

main / variables / outputs and folder layout



3 • Sharing & versioning

Registry, private git, and pinning versions.



4 • Lab 09

Refactor your Azure project into modules

01

What is a Module?

A container for resources used together — code reuse, done right.

Modules & the orchestrator pattern

A module groups related resources. Your root `main.tf` becomes an **orchestrator** that calls modules.

- Brings DRY to infrastructure
- Separation of concerns & modularity
- Source can be local or remote

root module (main.tf)
the orchestrator



network

app_service

Benefits & structure

Reusability

Write once, use everywhere.

Organization

Keep configs manageable.

Collaboration

Share infrastructure patterns.

TEXT

```
modules/  
└─ network/  
    ├── main.tf      # resources  
    ├── variables.tf # inputs  
    └─ outputs.tf    # outputs
```

HCL

```
module "network" {  
  source    = "../modules/network"  
  prefix    = local.prefix  
  location = var.location  
}
```

02

Sharing Modules

From the public registry to your private git repos.

Registry & private sharing



Terraform Registry

Prebuilt, community & vendor-verified modules at `registry.terraform.io` — used via `source`.



Private git

Point `source` at a git repo — needs auth on machine. Great for internal patterns.

⚠ Verified modules speed you up but cost you control of the underlying code — a real trade-off

Versioning & best practices

- Registry → set `version`
- Git → pin a tag, branch or commit SHA
- Keep modules focused & single-purpose
- Meaningful variable names & defaults
- Document inputs and outputs

HCL

```
module "network" {  
  source = "Azure/network/azurerem"  
  version = "~> 5.0"  
}  
  
module "internal" {  
  source = "git::https://github.com/org/mod"}
```

 LIVE DEMO

Create & use a module

Extract a resource group / network module, call it from the root, wire inputs & outputs, then consume a shared registry module.

Demo · 09 **Provider** · azurerm

📖 LAB 09 · 30-40 MIN

Refactor your project into module

Guided: extract the **network** module together, then modularize **app_service** and **psql** on your own. Reference Demo 09 if you get stuck.

You'll practise: module structure · inputs/outputs · `module` blocks · re-apply with no infra cha

Key takeaways

✓ **Modules = reuse**

Root main.tf orchestrates focused modules.

✓ **Structure is simple**

main / variables / outputs, clear inputs.

✓ **Always version**

Registry version or git ref — never floating.

✓ **Next up**

Session 10 — Terraform in CI/CD.



Questions?

The finale — Session 10: Terraform in
CI/CD.